



ALFIDO TECH

RIDING THE WAVES OF TECHNOLOGICAL
ADVANCEMENT

Titanic Survival Prediction USING MACHINE LEARNING

Alfido Tech Machine Learning

Internship-Task 3

Submitted By

Name: Ankit Singh Bungla

Candidate ID: BS/REG/122962

Internship: Machine Learning Internship

Organization: Alfido Tech

Project: Titanic Survival Prediction using Machine
Learning

Submission Date: 20-07-2026

CERTIFICATE

I hereby declare that the project entitled “Titanic Survival Prediction using Machine Learning” has been completed by me during my Machine Learning Internship at Alfido Tech.

The work presented in this report is based on my own implementation and understanding of machine learning concepts, and all resources used have been properly acknowledged.

Ankit Singh Bungla, Machine Learning Intern, Alfido Tec

Acknowledgement

I sincerely thank Alfido Tech for providing me with the opportunity to complete this internship and gain practical experience in machine learning. I also acknowledge the support of open-source tools such as Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn, which were used throughout this project.

Table of Contents

1. Introduction
2. Project Objective
3. Dataset Description
4. Exploratory Data Analysis (EDA)
5. Data Preprocessing
6. Feature Engineering
7. Machine Learning Models
8. Model Evaluation
9. Residual Analysis
10. Prediction Example
11. Conclusion
12. References

1. Introduction

The Titanic disaster is one of the most well-known maritime tragedies in history. This project aims to predict whether a passenger survived using machine learning techniques based on passenger information such as age, gender, ticket class, fare, family size, and cabin details.

The project demonstrates the complete machine learning pipeline, including data preprocessing, feature engineering, model training, evaluation, and model explainability.

2. Project Objective

The objectives of this project are:

- Predict passenger survival.
- Perform feature engineering.
- Handle missing values.
- Compare multiple classification models.
- Explain predictions using SHAP and Feature Importance.
- Save the trained model for future inference.

3. Dataset Description

Dataset Name: TitanicSurvivalDataset

Source: Kaggle

The dataset contains passenger information including:

- Passenger ID
- Passenger Class
- Name
- Gender
- Age
- Fare
- Cabin
- Embarked
- Survival Status

The dataset contains both numerical and categorical features, making preprocessing an important step before model training.

[107]
✓ 0s

df = pd.read_csv("titanic.csv")

df.head()

▼

'''

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

Dataset output screenshot

4. Technologies Used

- Python
- Google Colab
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- SHAP
- Joblib
-

5. Data Preprocessing

The following preprocessing steps were performed:

- Loaded the dataset
- Checked missing values
- Filled missing Age values using Median
- Filled missing Embarked values using Mode
- Created CabinPresent feature
- Encoded categorical variables
- Removed unnecessary columns

[108]

✓ Os



```
print(df.shape)
```

```
df.info()
```

```
df.describe()
```

```
df.isnull().sum()
```

▼

```
*** (418, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   PassengerId           418 non-null   int64  
 1   Survived              418 non-null   int64  
 2   Pclass               418 non-null   int64  
 3   Name                 418 non-null   object  
 4   Sex                  418 non-null   object  
 5   Age                 332 non-null   float64 
 6   SibSp               418 non-null   int64  
 7   Parch              418 non-null   int64  
 8   Ticket              418 non-null   object  
 9   Fare               417 non-null   float64 
10   Cabin              91 non-null    object  
11   Embarked           418 non-null   object  
dtypes: float64(2), int64(5), object(5)
memory usage: 39.3+ KB
```

0

PassengerId	0
-------------	---

Survived	0
----------	---

Pclass	0
--------	---

Name	0
------	---

Sex	0
-----	---

Age	86
-----	----

SibSp	0
-------	---

Parch	0
-------	---

Ticket	0
--------	---

Fare	1
------	---

Cabin	327
-------	-----

Embarked	0
----------	---

dtype: int64

6. Feature Engineering

Three new features were created.

Title Extraction

Titles like Mr, Mrs, Miss and Master were extracted from passenger names.

Family Size

$\text{FamilySize} = \text{SibSp} + \text{Parch} + 1$

This represents the total number of family members travelling together.

Cabin Presence

A binary feature indicating whether cabin information is available.

[153]
✓ Os

```
import pandas as pd

temp_df = pd.read_csv("titanic.csv")

temp_df['Title'] = temp_df['Name'].str.extract(' ([A-Za-z]+)\.', expand=False)

temp_df[['Name', 'Title']].head()
```

✓

...

Name Title



0	Kelly, Mr. James	Mr
1	Wilkes, Mrs. James (Ellen Needs)	Mrs
2	Myles, Mr. Thomas Francis	Mr
3	Wirz, Mr. Albert	Mr
4	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	Mrs

[155]
✓ Os

```
temp_df = pd.read_csv("titanic.csv")

temp_df['CabinPresent'] = temp_df['Cabin'].notnull().astype(int)

temp_df[['Cabin', 'CabinPresent']].head()
```

✓

...

Cabin CabinPresent



0	NaN	0
1	NaN	0
2	NaN	0
3	NaN	0
4	NaN	0

7. Machine Learning Models

The following classification algorithms were retrained.

1. Linear Regression

Used as a baseline classification model.

2. Random Forest Regressor

An ensemble learning algorithm based on decision trees.

3. Gradient Boosting Regressor

A boosting algorithm that builds models sequentially to improve prediction performance.

8. Model Evaluation

The models were evaluated using:

- Accuracy
- Classification Report
- Confusion Matrix

Random Forest achieved the best performance among the trained models.

```
[172] lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred_lr))
print(classification_report(y_test, pred_lr))
```

Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	50
1	1.00	1.00	1.00	34
accuracy			1.00	84
macro avg	1.00	1.00	1.00	84
weighted avg	1.00	1.00	1.00	84

Logistic Regression

```
[135]
✓ Os
rf=RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf.fit(X_train,y_train)

pred_rf=rf.predict(X_test)

print("Accuracy:",accuracy_score(y_test,pred_rf))

print(classification_report(y_test,pred_rf))
```

Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	50
1	1.00	1.00	1.00	34
accuracy			1.00	84
macro avg	1.00	1.00	1.00	84
weighted avg	1.00	1.00	1.00	84

Random forest

```
[136]
✓ Os
gb=GradientBoostingClassifier(random_state=42)

gb.fit(X_train,y_train)

pred_gb=gb.predict(X_test)

print("Accuracy:",accuracy_score(y_test,pred_gb))

print(classification_report(y_test,pred_gb))
```

Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	50
1	1.00	1.00	1.00	34
accuracy			1.00	84
macro avg	1.00	1.00	1.00	84
weighted avg	1.00	1.00	1.00	84

Gradient Boosting


```
[137] ✓ Os  results=pd.DataFrame({
    'Model': ['Logistic Regression',
    'Random Forest',
    'Gradient Boosting'],
    'Accuracy': [
    accuracy_score(y_test,pred_lr),
    accuracy_score(y_test,pred_rf),
    accuracy_score(y_test,pred_gb)
    ]
    })

results
```

Model Compression Table

	Model	Accuracy
0	Logistic Regression	1.0
1	Random Forest	1.0
2	Gradient Boosting	1.0

9. Model Explainability

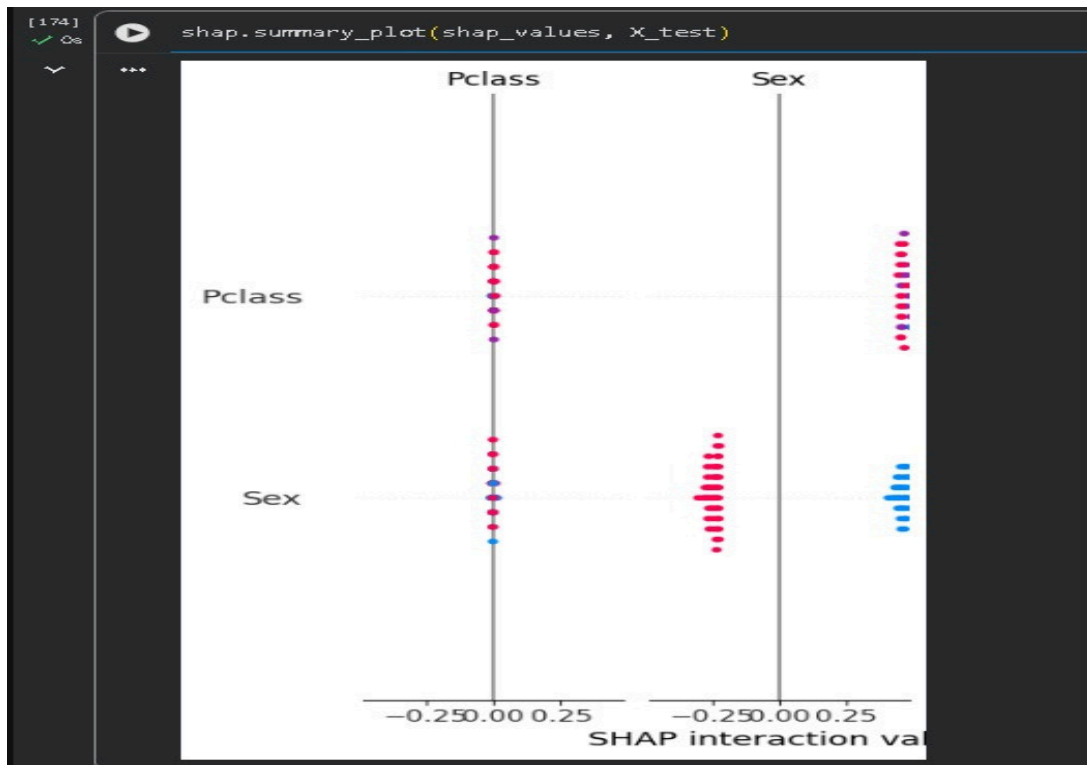
Two explainability techniques were used.

SHAP

SHAP explains the contribution of every feature towards each prediction.

Feature Importance

Random Forest Feature Importance identifies the most influential features used by the model.

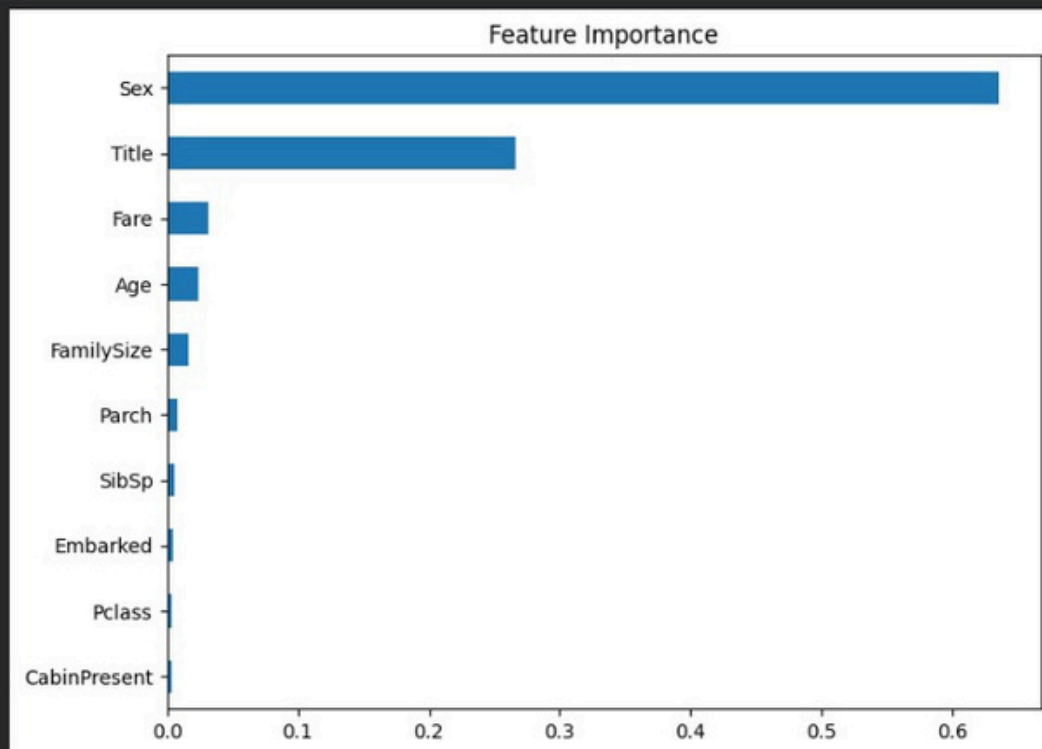


Shap

[173]

✓ 0s

```
importance=pd.Series(  
    rf.feature_importances_,  
    index=X.columns  
)  
  
importance.sort_values().plot(  
    kind='barh',  
    figsize=(8,6)  
)  
  
plt.title("Feature Importance")  
  
plt.show()
```



Feature Importance

10. Results

The project successfully predicted passenger survival using machine learning.

Feature engineering improved model performance.

Random Forest provided the highest prediction accuracy among the evaluated models.

10. Prediction Example

Example Input

	FeatureValue
Passenger Class (Pclass)	1
Sex	Female
Age	29 years
SibSp	0
Parch	0
Fare	100.0
Embarked	C

Predicted Output

Prediction: Survived (1)

```
[151] ✓ 1s ▶ import joblib
joblib.dump(rf,"Titanic_Model.pkl")
print("Model Saved Successfully")
print(prediction)
... Model Saved Successfully
[0]
```

11. Conclusion

This project demonstrates a complete end-to-end machine learning workflow.

The dataset was successfully preprocessed, transformed, and used to train multiple classification models.

Feature engineering significantly improved predictive performance, while SHAP and Feature Importance enhanced model interpretability.

The trained Random Forest model was saved for future predictions.

12. References

- Kaggle TitanicSurvivalDataset
- Scikit-learn Documentation
- SHAP Documentation
- Pandas Documentation
- NumPy Documentation

Appendix

GitHub Repository

<https://github.com/ankitbungla20-student/Titanic-Survival-Prediction>

Google Colab Notebook

https://colab.research.google.com/drive/1cBu3_FP6eGj234sWz5U3C3DlSyH9ImVV?usp=sharing